

Aula 16 - Refatoração e Padrões de Projeto em Java

Duração: 2 horas

Objetivos:

- Identificar code smells no projeto
- Aplicar padrões de projeto para melhorar a estrutura
- Realizar refatoração segura com testes automatizados

Parte Teórica (30 min)

1. Code Smells Comuns

Smell	Padrão Recomendado	Sintomas
Long Method	Strategy/Extract Method	Método > 10 linhas
Primitive Obsession	Builder/Value Object	Uso excessivo de tipos primitivos
Feature Envy	Move Method	Método acessa mais outros objetos que o próprio

2. Padrões Úteis para Refatoração

- **Strategy:** Variações de algoritmos (ex: diferentes cálculos de frete)
- **Factory:** Criação complexa de objetos (ex: diferentes tipos de usuários)
- **Decorator:** Adicionar comportamentos dinamicamente (ex: logs, cache)

Atividade Prática (1h30min)

Cenário: Refatorar Serviço de Pedidos

Código Original (Problemático)

```
java
// Anti-pattern: Long Method + Switch Statement
public class PedidoService {
```

```

public BigDecimal calcularTotal(Pedido pedido, String tipoCliente) {
    BigDecimal total = BigDecimal.ZERO;

    // Cálculo dos itens
    for (ItemPedido item : pedido.getItems()) {
        total = total.add(item.getPreco().multiply(new BigDecimal(item.getQuantidade
    ))));
    }

    // Cálculo de descontos
    switch (tipoCliente) {
        case "VIP":
            total = total.multiply(new BigDecimal("0.9")); // 10% off
            break;
        case "PREMIUM":
            total = total.multiply(new BigDecimal("0.8")); // 20% off
            break;
        // ... mais casos
    }

    // Cálculo de frete
    if (total.compareTo(new BigDecimal("100")) < 0) {
        total = total.add(new BigDecimal("15")); // Frete fixo
    }

    return total;
}
}

```

Passo 1: Aplicar Strategy Pattern

1. Interface de Desconto

java

```

public interface PoliticaDesconto {
    BigDecimal aplicarDesconto(BigDecimal total);
}

// Implementações:
public class DescontoVIP implements PoliticaDesconto {
    public BigDecimal aplicarDesconto(BigDecimal total) {
        return total.multiply(new BigDecimal("0.9"));
    }
}

```

2. Serviço Refatorado

java

```
public class PedidoService {
    private final PoliticaDesconto descontoStrategy;
    private final CalculadoraFrete calculadoraFrete;

    // Injeção de dependências
    public PedidoService(PoliticaDesconto strategy, CalculadoraFrete frete) {
        this.descontoStrategy = strategy;
        this.calculadoraFrete = frete;
    }

    public BigDecimal calcularTotal(Pedido pedido) {
        BigDecimal total = pedido.getItems().stream()
            .map(item -> item.getPreco().multiply(BigDecimal.valueOf(item.getQuantidade
            ())))
            .reduce(BigDecimal.ZERO, BigDecimal::add);

        total = descontoStrategy.aplicarDesconto(total);
        return calculadoraFrete.calcular(total);
    }
}
```

Passo 2: Escrever Testes para a Nova Implementação

Teste Unitário

java

```
class PedidoServiceTest {
    @Test
    void deveCalcularTotalComDescontoVIP() {
        PoliticaDesconto strategy = new DescontoVIP();
        CalculadoraFrete freteMock = mock(CalculadoraFrete.class);
        when(freteMock.calcular(any())).thenReturn(new BigDecimal("90"));

        PedidoService service = new PedidoService(strategy, freteMock);
        Pedido pedido = new Pedido(List.of(new ItemPedido(new BigDecimal("100"), 1)));

        assertEquals(new BigDecimal("90"), service.calcularTotal(pedido));
    }
}
```

Passo 3: Verificar Impacto no SonarQube

1. Execute a análise:

```
bash
```

```
mvn sonar:sonar
```

2. Compare as métricas antes/depois:

- Complexidade ciclomática reduzida
- Número de code smells diminuído

Checklist de Entrega

- Padrão Strategy aplicado em pelo menos 1 classe
- Testes atualizados para nova implementação
- PR aberto com título "[REFACTOR] Padrão Strategy em PedidoService"

Tarefa Pós-Aula

1. Identifique outro code smell no projeto e aplique:

- **Factory Method** para criação de usuários
- **Decorator** para adicionar logging

2. Documente no `docs/decisions.md`:

```
markdown
```

```
## Decisão 2024-05-20
**Padrão:** Strategy
**Motivo:** Remover switch statement complexo
**Arquivos alterados:**
- PedidoService.java
- PoliticaDesconto.java
```

Próximos Passos

- Aula 17: Integração com Frontend (Thymeleaf/API REST)

- **Aula 18:** Configuração de Monitoramento

Dica do Expert:

"Use `@Conditional` do Spring para selecionar estratégias em runtime!"

Recursos:

- [Refactoring Guru - Strategy](#)
- [Spring @Conditional](#)

Transforme código legado em código limpo! 🛠️ ✨